

Dokumentacja projektu

System zarządzania szpitalem (Baza danych)

Przedmiot

Programowanie serwera baz danych
Rok akademicki 2025/2026

Autorzy

Maja Adrianna

Github page

Link: <https://adrianna41.github.io/Szpital/>

Spis treści

1. Cel projektu
2. Funkcjonalności realizowane przez system
3. Diagram przypadków użycia
4. Diagram ERD
5. Opis encji
6. Opis relacji w bazie danych
7. Aspekty, których projekt nie obejmuje
8. Słownik danych
9. Uzasadnienie najważniejszych decyzji projektowych
10. Model architektury systemu
11. Opis implementacji
12. Opis zapytań, procedur i funkcji
13. Transakcje
14. Opis mechanizmów bezpieczeństwa i administracji

Cel projektu

Baza danych Szpital ma zostać użyta do aplikacji do zarządzania szpitalem, która ma na celu wspierać zarówno lekarzy w ich codziennej pracy, jak i umożliwić pacjentom umawianie wizyt oraz sprawdzanie swoich danych medycznych, a także ma umożliwiać planowanie grafików sal i wizyt oraz rejestrować dane o pracownikach i pacjentach.

Funkcjonalności realizowane przez aplikację, która opiera się na bazie danych Szpital:

- rejestracja i logowanie pacjenta
- dodawanie nowych pracowników przez administratora systemu
- umówienie się pacjenta na wizytę do specjalisty
- wyświetlenie danych pacjenta (na jego koncie) m.in informacji dotyczących wizyt, chorób, historii hospitalizacji, skierowań i recept
- tworzenie recept, diagnoz i skierowań przez lekarzy
- wyświetlanie statystyk, informacji o chorobach
- ustalanie grafików sal/wizyt przez administratora

Diagram przypadków użycia systemu szpitala:

Plik w formacie Visio: [use_case_szpital.vsd](#)

Plik w formacie png: [use_case_szpital.png](#)

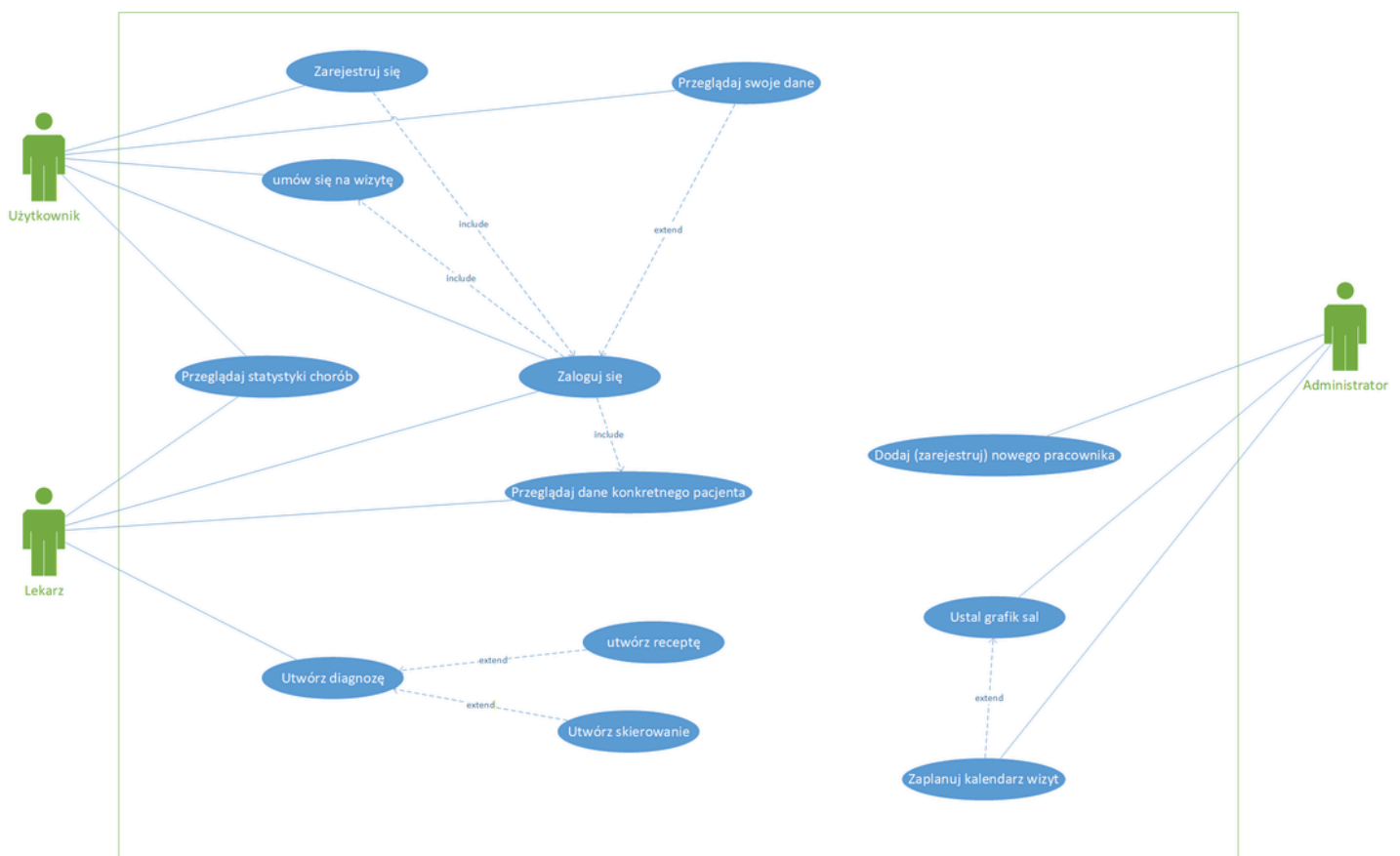
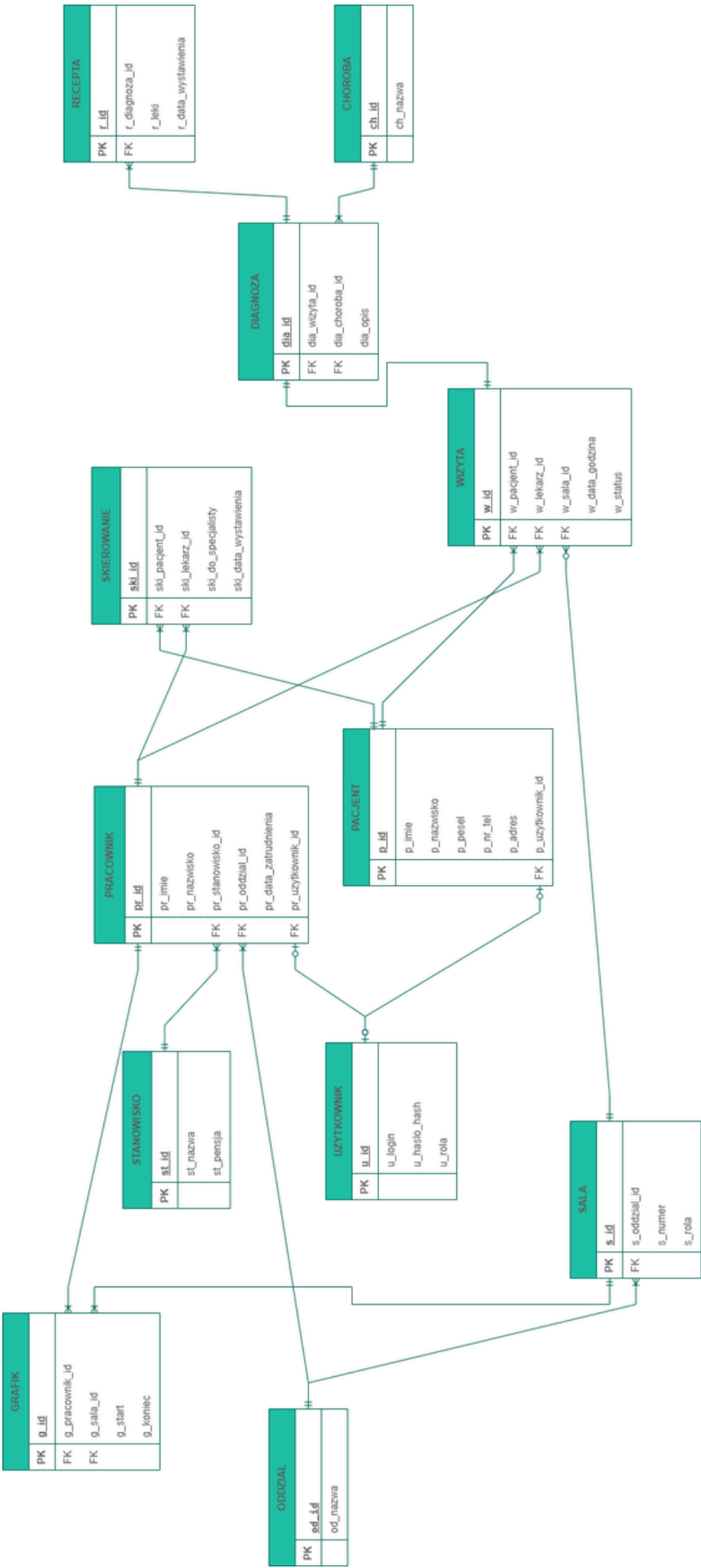


Diagram ERD bazy danych:

Plik drawio: [diagram_erd.drawio](#)
Plik png: [diagram_erd_szpital.png](#)



Baza danych (SQL Server Management Studio)

Skrypt tworzący strukturę tabel w bazie danych Szpital został wykonany w SQL Server Management Studio.

Plik: [tworzenie_tabel.sql](#)

Rdzeniem bazy jest tabela WIZYTA, ponieważ celem szpitala jest przyjmowanie pacjentów na wizyty w celu weryfikacji ich zdrowia. Encja ta jest powiązana z encjami PACJENT, PRACOWNIK i SALA a także tylko dzięki zrealizowanej wizycie jest możliwe stawianie diagnoz i wystawienie recept przez lekarzy.

Opis encji w bazie danych Szpital:

- ODDZIAL - zawiera nazwy wszystkich oddziałów szpitala
- STANOWISKO - tabela przechowująca informacje o nazwach stanowisk w szpitalu i pensjach, które odpowiadają danemu stanowisku
- CHOROBA - zawiera nazwy chorób
- SALA - posiada informacje o pomieszczeniach w szpitalu: numer sali, jej rola oraz do jakiego oddziału jest przypisana
- UZYTKOWNIK - encja przechowuje dane do logowania użytkowników systemu: administratorów, lekarzy i pacjentów. Zawiera unikalny numer użytkownika, login, hash hasła oraz rolę przypisaną użytkownikowi.
- PRACOWNIK - przechowuje informacje o pracownikach szpitala: numer identyfikacyjny pracownika, imię, nazwisko, obejmowane stanowisko, oddział, do którego pracownik jest przypisany, datę zatrudnienia, numer użytkownika systemu (jeśli stanowisko pracownika to administrator lub lekarz)
- PACJENT - encja przechowująca dane pacjentów: numer identyfikacyjny pacjenta, imię, nazwisko, numer PESEL, numer telefonu, adres, numer użytkownika systemu
- WIZYTA - tabela zawierająca informacje o umówionych wizytach: numer pacjenta, który umówił wizytę, numer lekarza przypisanego do wizyty, numer sali (jeśli wizyta nie jest konsultacją zdalną), datę i godzinę wizyty oraz jej status
- GRAFIK - encja, która odpowiada za przypisywanie godzin pracy pracownikom w poszczególne dni. Zawiera atrybuty: numer identyfikacyjny pracownika, identyfikator sali, data i godzina rozpoczęcia pracy, data i godzina zakończenia pracy pracownika
- SKIEROWANIE - przechowuje dane z wystawionych przez lekarzy skierowań: dane pacjenta, który otrzymał skierowanie, lekarza, który wydał skierowanie oraz informacje do jakiego specjalisty pacjent powinien się udać, a także datę wystawienia skierowania
- DIAGNOZA - umieszczane są w niej dane o diagnozach pacjentów, zawiera informacje o nazwie choroby, wizycie, na której postawiono diagnozę oraz opis diagnozy.
- RECEPTA - przechowuje recepty, które zostały wystawione przez lekarzy. Jest powiązana z diagnozą i zawiera informacje o lekach, które pacjent powinien wykupić oraz posiada datę wystawienia.

Opis relacji w bazie danych

Relacje 1:N (Jeden do wielu):

- Pracownik (Lekarz) może przeprowadzić wiele wizyt, ale jedna wizyta jest prowadzona przez jednego Lekarza.
- Pacjent może odbyć wiele Wizyt, ale konkretna Wizyta dotyczy tylko jednego pacjenta.
- Wizyta może skutkować wydaniem kilku diagnoz (np. gdy pacjent ma choroby współistniejące).

Relacje 1:1 (Jeden do jednego):

- Użytkownik systemu jest powiązany z konkretnym pracownikiem lub pacjentem.
Oddzielenie tabeli UZYTKOWNIK od danych osobowych ułatwia zarządzanie autoryzacją.

Aspekty, których projekt nie obejmuje:

- Modułu księgowego i finansowego (rozliczenia z NFZ, fakturowanie).
- Modułu magazynowego apteki szpitalnej (śledzenia dokładnych stanów magazynowych leków).
- Zintegrowanego systemu laboratoryjnego (szczegółowych wyników badań krwi, obrazowych itp.).

Słowniki danych:

Tabela ODDZIAL

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	Krótki opis znaczenia pola
od_id	INT	PK	NOT NULL	IDENTITY(1,1)	Identyfikator oddziału
od_nazwa	NVARCHAR(100)	-	NOT NULL	-	Nazwa oddziału szpitala

Tabela STANOWISKO

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
st_id	INT	PK	NOT NULL	IDENTITY(1,1)	Identyfikator stanowiska
st_nazwa	NVARCHAR(50)	-	NOT NULL	-	Nazwa stanowiska
st_pensja	DECIMAL(10, 2)	-	NULL	-	Podstawowa pensja, jaką otrzymuje pracownik szpitala na danym stanowisku

Tabela CHOROBA

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
ch_id	INT	PK	NOT NULL	IDENTITY(1,1)	Numer identyfikacyjny choroby w szpitalu
ch_nazwa	NVARCHAR(100)	-	NOT NULL	-	Nazwa choroby

Tabela SALA

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
s_id	INT	PK	NOT NULL	IDENTITY(1,1)	Numer identyfikacyjny sali
s_oddzial_id	INT	FK	NOT NULL	FK: ODDZIAL(od_id)	Powiązanie z identyfikatorem oddziału z encji ODDZIALY
s_numer	NVARCHAR(10)	-	NOL NULL	-	Numer sali
s_rola	NVARCHAR(50)	-	NULL	-	Przeznaczenie sali

Tabela UZYTKOWNIK

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
u_id	INT	PK	NOT NULL	IDENTITY(1,1)	Numer identyfikacyjny użytkownika aplikacji
u_login	NVARCHAR(50)	-	NOT NULL	UNIQUE	Login użytkownika
u_haslo_hash	NVARCHAR(255)	-	NOT NULL	-	Skrót hasła (hash)
u_rola	NVARCHAR(20)	-	NOT NULL	CHECK (u_rola IN ('admin', 'lekarz', 'pacjent'))	Rola użytkownika aplikacji (admin, lekarz, pacjent)

Tabela PRACOWNIK

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
pr_id	INT	PK	NOT NULL	IDENTITY(1,1)	Numer identyfikacyjny pracownika
pr_imie	NVARCHAR(50)	-	NOT NULL	-	Imię pracownika
pr_nazwisko	NVARCHAR(50)	-	NOT NULL	-	Nazwisko pracownika
pr_stanowisko_id	INT	FK	NOT NULL	FK: STANOWISKO(st_id)	Powiązanie pracownika ze stanowiskiem, jakie posiada, odwołanie do tabeli STANOWISKO
pr_oddzial_id	INT	FK	NOT NULL	FK: ODDZIAL(od_id)	Powiązanie z oddziałem z tabeli ODDZIAL, do którego przypisany jest pracownik
pr_data_zatrudnienia	DATE	-	NOT NULL	DEFAULT GETDATE()	Data zatrudnienia pracownika
pr_uzytkownik_id	INT	FK	NULL	FK: UZYTEKOWNIK(u_id), UNIQUE	Unikalny numer identyfikacyjny użytkownika systemu, powiązanie z tabelą UZYTEKOWNIK

Tabela PACJENT

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
p_id	INT	PK	NOT NULL	IDENTITY(1,1)	Identyfikator pacjenta
p_imie	NVARCHAR(50)	-	NOT NULL	-	Imię pacjenta
p_nazwisko	NVARCHAR(50)	-	NOT NULL	-	Nazwisko pacjenta
p_pesel	NVARCHAR(11)	-	NOT NULL	UNIQUE	Numer PESEL pacjenta
p_nr_tel	NVARCHAR(15)	-	NULL	-	Numer telefonu pacjenta
p_adres	NVARCHAR(300)	-	NULL	-	Adres zamieszkania pacjenta
p_uzytkownik_id	INT	FK	NULL	REFERENCES UZYTKOWNIK(u_id)	Powiązanie z kontem użytkownika aplikacji szpitala

Tabela WIZYTA

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
w_id	INT	PK	NOT NULL	IDENTITY(1,1)	Numer identyfikacyjny wizyty
w_pacjent_id	INT	FK	NOT NULL	REFERENCES PACJENT(p_id)	Identyfikator pacjenta zapisanego na wizytę – odwołanie do tabeli PACJENT
w_lekarz_id	INT	FK	NOT NULL	REFERENCES PRACOWNIK(pr_id)	Identyfikator lekarza, do którego umówiona została wizyta – odwołanie do tabeli PRACOWNIK
w_sala_id	INT	FK	NULL	REFERENCES SALA(s_id)	Identyfikator sali, w której będzie odbywać się wizyta – odwołanie do tabeli SALA
w_data_godzina	DATETIME	-	NOT NULL	-	Data i godzina umówionej wizyty
w_status	NVARCHAR(20)	-	NULL	DEFAULT 'umówiona'	Status wizyty

Tabela GRAFIK

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
g_id	INT	PK	NOT NULL	IDENTITY(1,1)	Identyfikator wpisów w grafiku
g_pracownik_id	INT	FK	NOT NULL	REFERENCES PRACOWNIK(pr_id)	Pracownik pełniący dyżur.
g_sala_id	INT	FK	NOT NULL	REFERENCES SALA(s_id)	Pomieszczenie, w którym będzie pracował pracownik
g_start	DATETIME	-	NOT NULL	-	Data i godzina rozpoczęcia dyżuru.
g_koniec	DATETIME	-	NOT NULL	-	Data i godzina zakończenia dyżuru.

Tabela SKIEROWANIE

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
ski_id	INT	PK	NOT NULL	IDENTITY(1,1)	Unikalny identyfikator skierowania
ski_pacjent_id	INT	FK	NOT NULL	REFERENCES PACJENT(p_id)	Pacjent, któremu wystawiono skierowanie – powiązanie z tabelą PACJENT
ski_lekarz_id	INT	FK	NOT NULL	REFERENCES PRACOWNIK(pr_id)	Lekarz, który wystawił skierowanie – powiązanie z tabelą PRACOWNIK
ski_do_specjalisty	NVARCHAR(100)	-	NOT NULL	-	Oddział lub lekarz określonej specjalizacji, do którego pacjent powinien się udać
ski_data_wystawienia	DATE	-	NOT NULL	DEFAULT GETDATE()	Data wystawienia skierowania

Tabela DIAGNOZA

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
dia_id	INT	PK	NOT NULL	IDENTITY(1,1)	Identyfikator diagnozy
dia_wizyta_id	INT	FK	NOT NULL	REFERENCES WIZYTA(w_id)	Wizyta, podczas której, dokonano postawienia diagnozy – powiązanie z tabelą WIZYTA
dia_choroba_id	INT	FK	NOT NULL	REFERENCES CHOROBA(ch_id)	Choroba, która zdiagnozowano – powiązanie z tabelą CHOROBA
dia_opis	NVARCHAR(2500)	-	NULL	-	Opis lekarski, zalecenia, profilaktyka.

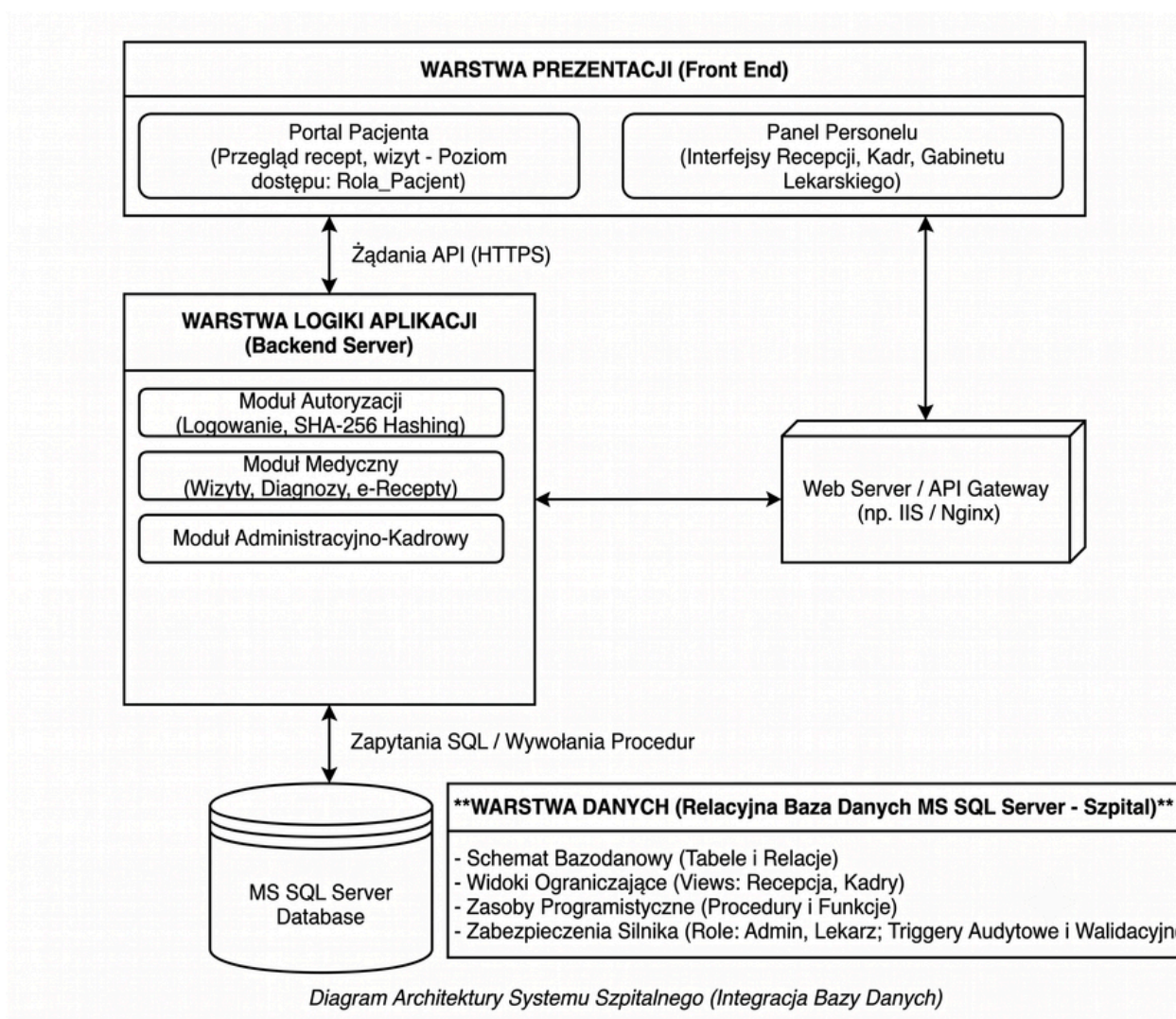
Tabela RECEPTA

Nazwa kolumny	Typ danych	Klucz główny lub obcy	Wymagalność wartości	Ograniczenia i reguły walidacji	krótki opis znaczenia pola
r_id	INT	PK	NOT NULL	IDENTITY(1,1)	Identyfikator recepty
r_diagnoza_id	INT	FK	NOT NULL	REFERENCES DIAGNOZA(dia_id)	Odwołanie do diagnozy choroby, której dotyczy recepta – tabela DIAGNOZA
r_leki	NVARCHAR(1300)	-	NOT NULL	-	Leki, na które wystawiono receptę oraz ich dawkowanie
r_data_wystawienia	DATE	-	NOT NULL	DEFAULT GETDATE()	Data wystawienia recepty

Uzasadnienie najważniejszych decyzji projektowych

1. Autogenerowanie kluczy (IDENTITY): Do wszystkich kluczy głównych użyto autoincrementacji (IDENTITY(1,1)). Uniezależnia to bazę od logiki aplikacji i eliminuje ryzyko duplikatów przy zbiegach operacji INSERT.
2. Tabele audytowe i logi: Zamiast nadpisywać i tracić historię edycji, zaprojektowano osobne tabele logów (np. WIZYTY_LOGI, PensjaAudit). Obsługiwane są one przez Triggery bazodanowe (wyzwalacze). Pozwala to śledzić zmiany wrażliwych danych medycznych.
3. Standaryzacja danych (Słowniki): Takie dane jak nazwy stanowisk, nazwy oddziałów czy schorzenia wyniesiono do osobnych tabel słownikowych. Zapobiega to literówkom i niespójnościom.
4. Więzy integralności i defaults: Użyto constraintów typu UNIQUE np. dla numeru PESEL oraz wartości domyślnych (DEFAULT GETDATE()) dla dat, co zapewnia podstawową walidację.

Model architektury systemu:



Wstawianie danych do tabel

Plik: [dodanie_rekordow.sql](#)

Wstawianie rekordów zostało podzielone na cztery etapy:

1. Tabele ODDZIAŁ, STANOWISKO, CHOROBA, SALA - słownikowe:

Dane zostały dodane za pomocą składni INSERT INTO.

2. Tabele PRACOWNIK i PACJENT

Na początku zostały utworzone pomocnicze funkcje tablicowe (fn_ListaImion(), fn_ListaNazwisk(), fn_ListaImionZ(), fn_ListaNazwiskZ()) przechowujące odpowiednio po 50: imion męskich, nazwisk męskich, imion żeńskich, nazwisk żeńskich. Następnie za pomocą procedur GenerujPracownikow, GenerujPacjentow w losowy sposób wygenerowani zostali pracownicy. Wykorzystywana została pętla WHILE oraz obsługa błędów TRY/CATCH do wielokrotnego powtarzania operacji wstawiania INSERT. W każdej iteracji procedura losuje płeć, pobiera odpowiednie imię i nazwisko z wyżej wymienionych funkcji tablicowych, autogeneruje unikalny 11-cyfrowy numer PESEL oraz losowy format numeru telefonu.

3. Tabela UZYTKOWNICY

Dane kont użytkowników zostały wygenerowane za pomocą zautomatyzowanej procedury składowanej UzupełnijKontaPracownikow, wykorzystującej mechanizm kursora. Procedura ta iteruje po pracownikach wyższego szczebla (lekarzach, ordynatorach i dyrekcji), którzy nie posiadają jeszcze przypisanego konta. Dla każdej takiej osoby skrypt dynamicznie generuje unikalny login (w formacie imię.nazwisko, z obsługą duplikatów), szyfruje domyślne hasło algorytmem SHA-256 oraz nadaje odpowiednią rolę systemową (admin dla dyrekcji, lekarz dla pozostałych). Na sam koniec procedura automatycznie aktualizuje tabelę PRACOWNIK, powiązując nowo utworzone konto z konkretną osobą, co gwarantuje pełną spójność kluczy obcych.

4. Tabele transakcyjne WIZYTA, DIAGNOZA, RECEPTA, SKIEROWANIE

Ich rekordy zostały wstawione za pomocą polecenia INSERT INTO.

Skrypt inicjalizujący bazę danych

Plik: [szpital_skrypt_inicjalizujacy.sql](#)

Plik zawiera skrypt inicjalizujący bazę danych Szpital. Po uruchomieniu go w Microsoft SQL Server utworzą się tabele oraz zostaną wypełnione danymi. Skrypt został wygenerowany za pomocą narzędzia Microsoft SQL Server na utworzonej bazie danych Szpital.

Więzy integralności

PW celu ochrony spójności bazy danych na poziomie silnika, poza standardowymi relacjami kluczy obcych (FOREIGN KEY), zaimplementowano szereg więzów integralności.

Najważniejsze z nich to:

- Więzy unikalności - UNIQUE: Nałożone na kolumnę p_pesel w tabeli PACJENT oraz u_login w tabeli UZYTKOWNIK, zapobiegające duplikacji kont i danych osobowych.
- Więzy sprawdzające - CHECK: Wymuszające poprawne wartości logiczne, np. w tabeli UZYTKOWNIK upewniające się, że rola to tylko dopuszczalne wartości: CHECK (u_rola IN ('admin', 'lekarz', 'pacjent')).
- Wartości domyślne - DEFAULT: Automatyzujące wprowadzanie danych, np. DEFAULT GETDATE() dla kolumn z datą wystawienia recepty lub skierowania.

Przykładowe dane testowe

Dzięki zastosowaniu procedur generujących, baza została wypełniona realistycznymi danymi testowymi. Poniżej znajduje się przykład wygenerowanych rekordów dla pacjentów

	p_id	p_imie	p_nazwisko	p_pesel	p_nr_tel	p_adres	p_uzytkownik_id
1	1	Janusz	Kamiński	16494875867	505-464-401	ul. Testowa 118, 90-001 Łódź	11
2	2	Beata	Michalska	22977460517	587-612-540	ul. Testowa 74, 90-001 Łódź	12
3	3	Alicja	Borkowska	82729789925	625-998-814	ul. Testowa 118, 90-001 Łódź	13
4	4	Mariola	Bąk	57952990749	706-253-900	ul. Testowa 56, 90-001 Łódź	14
5	5	Gabriela	Sadowska	90868786056	897-141-135	ul. Testowa 121, 90-001 Łódź	15
6	6	Halina	Wieczorek	27574220232	684-752-523	ul. Testowa 71, 90-001 Łódź	16
7	7	Zbigniew	Dąbrowski	31257016412	525-300-958	ul. Testowa 12, 90-001 Łódź	17
8	8	Przemysław	Krawczyk	54591567138	537-470-466	ul. Testowa 15, 90-001 Łódź	18
9	9	Ewelina	Król	82197296300	531-410-519	ul. Testowa 36, 90-001 Łódź	19
10	10	Artur	Dudek	39421155874	653-652-117	ul. Testowa 47, 90-001 Łódź	20
11	11	Joanna	Michalak	29523617609	620-600-220	ul. Testowa 49, 90-001 Łódź	21
12	12	Barbara	Lis	90234791817	829-577-231	ul. Testowa 149, 90-001 Łódź	22
13	13	Stanisław	Szewczyk	93221420719	893-223-273	ul. Testowa 22, 90-001 Łódź	23
14	14	Mariusz	Walczak	74550750241	527-508-359	ul. Testowa 142, 90-001 Łódź	24
15	15	Emil	Sadowski	36488269812	746-652-553	ul. Testowa 86, 90-001 Łódź	25
16	16	Bożena	Majewska	99975831776	537-211-131	ul. Testowa 54, 90-001 Łódź	26
17	17	Grażyna	Sikora	86539295570	862-689-549	ul. Testowa 37, 90-001 Łódź	27
18	18	Piotr	Zieliński	65589853339	891-906-928	ul. Testowa 7, 90-001 Łódź	28
19	19	Mikołaj	Kowalski	36115649936	855-495-297	ul. Testowa 102, 90-001 Łódź	29
20	20	Hamlet	Marciniak	61846637982	790-684-416	ul. Testowa 33, 90-001 Łódź	30

Kolejność uruchamiania skryptów

Aby poprawnie zainicjalizować strukturę bazy danych Szpital i uniknąć błędów związanych z brakiem wymaganych obiektów (np. próbą wstawienia danych do nieistniejącej tabeli), skrypty należy uruchamiać w następującej kolejności:

- 1.tworzenie_tabel.sql – utworzenie schematu bazy danych, definicja tabel, kluczy głównych i obcych.
- 2.dodanie_rekordow.sql – wywołanie procedur generujących dane i zasilenie tabel informacjami testowymi.
- 3.funkcja_skalarna.sql – utworzenie funkcji (fn_LiczbaZakonczonychWizytPacjenta), która będzie potrzebna do działania widoków i procedur.
- 4.perspektywy.sql – utworzenie widoków dla recepcji i kadr.
- 5.trigger_logi.sql oraz zaawansowane_triggery.sql – nałożenie triggerów audytowych i walidacyjnych na wypełnione tabele.
- 6.role.sql – utworzenie ról systemowych i nadanie uprawnień dla użytkowników.

Widoki

Plik: [perspektywy.sql](#)

W bazie Szpital zaimplementowano widoki systemowe (views). Pełnią one rolę wirtualnych tabel, które filtrują i łączą informacje z wielu miejsc, ukrywając jednocześnie dane wrażliwe.

- **Perspektywa Recepcji (vw_Recepcja_Wizyty):**

Jej celem jest usprawnienie pracy personelu na recepcji szpitala oraz ochrona danych medycznych pacjenta. Widok łączy tabele WIZYTA, PACJENT, PRACOWNIK oraz SALA. Wyświetla wyłącznie dane organizacyjne (imię i nazwisko pacjenta, PESEL, przydzielonego lekarza, numer sali oraz datę i status wizyty). Cechą kluczową tego widoku jest całkowite odcięcie pracownika recepcji od tabel medycznych – recepcjonista nie ma możliwości wglądu w diagnozy, schorzenia, wystawione recepty czy skierowania pacjenta.

- **Perspektywa Kadr (vw_Kadry_Zatrudnienie):**

Jej celem jest złączenie danych pracowniczych dla działu HR z jednoczesnym ukryciem mechanizmów autoryzacji. Widok zestawia informacje z tabel PRACOWNIK, STANOWISKO i ODDZIAL. Dodatkowo, w locie wykorzystuje funkcję skalarną fn_ObliczStazPracy, aby na bieżąco prezentować aktualny staż pracy każdego pracownika w latach. Zestawienie prezentuje stanowiska i pensje, ale celowo omija kolumny powiązane z tabelą UZYTKOWNIK. Dzięki temu dział kadr ma pełen obraz zatrudnienia, ale nie ma dostępu do loginów systemowych, ról ani zahaszowanych haseł personelu.

Role i uprawnienia

Plik: [role.sql](#)

Baza danych posiada trzy role systemowe:

- **Rola_Admin:** Przeznaczona dla personelu zarządzającego (dyrekcja, dział IT, kadry, recepcja). Posiada uprawnienia operacyjne (tworzenie kont UZYTKOWNIK, aktualizacja danych w tabeli PRACOWNIK i PACJENT), a także dostęp do widoków kadrowych i recepcyjnych. Rolę tę pozbawiono możliwości bezpośredniej modyfikacji danych medycznych (DENY INSERT, UPDATE, DELETE na tabelach DIAGNOZA czy RECEPTA).
- **Rola_Lekarz:** Profil o charakterze medycznym. Daje pracownikowi pełen dostęp operacyjny do tabel związanych z historią leczenia pacjenta (WIZYTA, DIAGNOZA, RECEPTA, SKIEROWANIE). Jednocześnie rola ta ma zablokowany dostęp do edycji danych kadrowych i zarządzania kontami innych użytkowników.
- **Rola_Pacjent:** Profil przeznaczony tylko do odczytu. Pozwala na tworzenie zapytań do bazy danych w celu wyświetlenia historii własnych wizyt, e-recept i e-skierowań. Za pomocą polecenia DENY całkowicie zablokowano pacjentom możliwość wykonywania operacji INSERT, UPDATE oraz DELETE na jakichkolwiek tabelach.

Zapytania DQL - raporty i agregacje

Plik: [raport.sql](#)

Plik: [grupowanie_danych.sql](#)

Plik: [okna_danych.sql](#)

W projekcie wykorzystane zostały następujące mechanizmy generowania raportów i agregacji:

- **Eksport do XML / JSON:** Zapytania z klauzulami FOR XML AUTO, ELEMENTS oraz FOR JSON PATH, generujące gotową historię wizyt pacjentów do formatów wymiany danych z systemami zewnętrznymi.
- **Agregacje i grupowanie:** Raport dotyczący analizy obciążenia placówki w czasie, który za pomocą funkcji YEAR() i MONTH() oraz instrukcji CASE zlicza zakończone wizyty w podziale na poszczególne miesiące.
- **Funkcje okna:** Zastosowane zostały zaawansowane funkcje analityczne: RANK() OVER do tworzenia rankingu wizyt danego pacjenta oraz LAG() wraz z DATEDIFF() do wyliczania liczby dni, jaka upłynęła od poprzedniej wizyty pacjenta w placówce.

Procedury składowane i funkcje

Plik: [dodanie_rekordow.sql](#)

Plik: [funkcja_skalarna.sql](#)

W projekcie Szpital wykorzystane zostały procedury składowane (GenerujPacjentow, GenerujPracownikow, UzupelnijKontaPracownikow) i funkcja skalarna:

- **Procedura GenerujPacjentow:** Jej celem jest zautomatyzowane generowanie testowych danych pacjentów (mock data). Skrypt losuje płeć, dobiera odpowiednie imię i nazwisko ze słownika oraz generuje unikalny PESEL i numer telefonu. Parametry wejściowe: @ile (INT) – określona przez użytkownika liczba rekordów do wygenerowania. Wynikiem działania jest wstawienie podanej liczby nowych wierszy bezpośrednio do tabeli PACJENT. Przykład użycia: `EXEC dbo.GenerujPacjentow 20;`
- **Procedura UzupelnijKontaPracownikow:** Jej celem jest generowanie poświadczeń logowania dla personelu. Skrypt iteruje po pracownikach wyższego szczebla za pomocą kursora, tworzy dla nich loginy i bezpiecznie szyfruje domyślne hasła przy użyciu algorytmu SHA-256. Nie posiada danych wejściowych, pobiera dane dynamicznie z bazy. Wynikiem jej działania jest wstawienie nowych rekordów do tabeli UZYTKOWNIK oraz zaktualizowanie kluczy obcych pr_uzytkownik_id w tabeli PRACOWNIK. Przykład użycia: `EXEC dbo.UzupelnijKontaPracownikow;`
- **Funkcja skalarna:** fn_LiczbaZakonczonychWizytPacjenta - zlicza ilość zrealizowanych wizyt przez pacjentów. Parametry wejściowe: @PacjentID (INT) – unikalny identyfikator pacjenta. Wynik działania: Wartość liczbową całkowitą (INT) określającą liczbę wizyt.

Transakcje i obsługa błędów

Plik: [transakcje.sql](#)

Plik: [transakcja_obsługa_bledow.sql](#)

Aby zapewnić integralność danych i odporność systemu na awarie, w bazie zaimplementowano transakcje oraz bloki strukturalnej obsługi błędów (TRY...CATCH). Chronią one bazę przed wprowadzaniem częściowych lub uszkodzonych informacji.

- **Transakcja z punktami zapisu:** Zastosowano mechanizm SAVE TRAN podczas jednoczesnego dodawania nowego oddziału (ODDZIAL) oraz powiązanych z nim sal szpitalnych z tabeli SALA. W przypadku wystąpienia błędu na etapie przypisywania sali (np. zły numer, błąd naruszenia klucza), system wycofuje się tylko do punktu zapisu. Dzięki temu oddział zostaje poprawnie zapisany w bazie, a wycofywana jest wyłącznie błędna sala. Zapobiega to całkowitemu przerwaniu długich procesów wprowadzania danych.
- **Bezpieczne przechwytywanie wyjątków i logowanie awarii:** Baza posiada transakcję wstawiającą nową wizytę lekarską, zabezpieczoną blokiem TRY...CATCH. Skrypt celowo symuluje błąd biznesowy (próba przypisania wizyty do nieistniejącego pacjenta o ID 99999, co narusza klucz obcy). Zamiast wyświetlać użytkownikowi komunikat błędu, system płynnie przechodzi do sekcji CATCH, sprawdza stan uszkodzenia transakcji za pomocą funkcji XACT_STATE(), wykonuje bezpieczny ROLLBACK, a na końcu zapisuje szczegóły problemu (datę oraz treść systemowego komunikatu z ERROR_MESSAGE()) do dedykowanej tabeli technicznej Bledy.
- **Transakcja powiązana:** Zaimplementowano skrypt realizujący proces zapisu nowej wizyty i przypisania do niej diagnozy w ramach jednego bloku BEGIN TRANSACTION. Skrypt ten testuje zachowanie bazy w przypadku naruszenia więzów spójności np. wstawienie diagnozy z kodem choroby 99999, który nie istnieje w słowniku CHOROBA. System działa zgodnie z zasadą "wszystko albo nic" (atomowość). Gdy wstawienie diagnozy kończy się błędem, skrypt płynnie przechodzi do bloku CATCH. Tam, po upewnieniu się, że transakcja jest nadal otwarta (za pomocą zmiennej systemowej @@TRANCOUNT > 0), następuje jej całkowite wycofanie (ROLLBACK). Oznacza to, że z bazy usuwana jest również sama wizyta (mimo że jej INSERT wykonał się poprawnie), co chroni system przed powstawaniem "pustych", osieroconych rekordów. Dodatkowo skrypt posiada również obsługę wyjątków.

Triggerzy

Plik: [trigger_logi.sql](#)

Plik: [zaawansowane_triggerzy.sql](#)

Baza zawiera następujące triggerzy:

- **Śledzenie zmian w harmonogramie - sprawdzanie modyfikacji wizyt:** Zaimplementowano trigger trg_WizytaUpdateLog działający w trybie AFTER UPDATE na tabeli WIZYTA. Pełni on funkcję automatycznego dziennika zdarzeń (logu). Skrypt wykorzystuje wirtualne tabele inserted (nowe dane) oraz deleted (stare dane), łącząc je i precyzyjnie porównując. Jeśli system wykryje, że zmianie uległy kluczowe informacje – takie jak status wizyty lub jej termin (w_data_godzina) – wyzwalacz w tle zapisuje historyczne i zaktualizowane wartości do dedykowanej tabeli WIZYTY_LOGI.

Triggerzy obejmujące śledzenie historii:

- **trg_StanowiskaPensjaAudit:** Monitoruje zmiany w tabeli STANOWISKO. Używając funkcji UPDATE(st_pensja), sprawdza, czy modyfikacja dotyczyła wynagrodzenia, a następnie zapisuje do tabeli PensjaAudit kwotę przed i po zmianie, aby zapobiec cichym, nieautoryzowanym podwyżkom.
- **trg_PacjentColumnsAudit:** Rejestruje modyfikacje danych osobowych w tabeli PACJENT.

Triggery odpowiadające za blokowanie błędnych danych: Działają jako INSTEAD OF INSERT, przejmują operację wstawiania i najpierw sprawdzają dane.

- **trg_WizytyValidation:** Wyświetla błąd, jeśli ktoś próbuje zaplanować wizytę w czasie, który już minął - podanie daty z przeszłości.
- **trg_StanowiskaInsertValidation:** Uniemożliwia próby zdefiniowania ujemnej pensji.
- **trg_ReceptyComplexControl:** Trigger sprawdzający e-recepty. Odstępuje od dodania recepty, jeśli brakuje na niej wypisanych leków, data jest z przyszłości lub wskazana diagnoza nie istnieje.

Logika biznesowa:

- **trg_BlockWizytaPatientChange:** Trigger chroni spójność historii medycznej. Pokazuje wyjątek, jeśli ktoś chce podmienić przypisanego pacjenta (w_pacjent_id) w istniejącej i zakończonej już wizycie.
- **trg_StanowiskaPensjaCorrection:** Zabezpiecza bazę przed nieskończoną pętlą za pomocą TRIGGER_NESTLEVEL(). Automatycznie koryguje i wyrównuje wstawianą pensję do pułapu najniższej krajowej (4300 zł), jeśli wpisano kwotę niższą.

Trigger serwerowy:

- **trg_DDL_Klinika_Audit:** Założony na poziomie całej bazy danych. Używając funkcji EVENTDATA(), przechwytuje zapytania tworzące nowe tabele i zapisuje ich strukturę oraz dokładny kod T-SQL do specjalnego logu XML. Jest narzędziem dla administratora.